

UNIVERSIDAD CARLOS III



Criptografía y seguridad

Parte 2:

Desarrollo de una aplicación que utilice criptografía

Grupo 2

CURSO 2025 - 2026

RAQUEL DEL VALLE OLIVARES → 100522214@alumnos.uc3m.es

Grupo 85, Ingeniería Informática

XIYA YE → 100522159@alumnos.uc3m.es

Grupo 85, Ingeniería Informática

ÍNDICE

1- PROPÓSITO DEL PROYECTO.....	2
2- UTILIDAD FIRMA DIGITAL, ALGORITMOS UTILIZADOS Y GESTIÓN Y ALMACENAMIENTO DE CLAVES Y FIRMAS.....	2
2.1- UTILIDAD FIRMA DIGITAL.....	2
2.2- ALGORITMOS UTILIZADOS.....	2
2.3- GESTIÓN Y ALMACENAMIENTO DE CLAVES Y FIRMAS.....	3
3- GENERACIÓN CERTIFICADOS DE CLAVE PÚBLICA, JERARQUÍA DE AUTORIDADES DE CERTIFICACIÓN E IMPLEMENTACIÓN.....	4
3.1- GENERACIÓN CERTIFICADOS CLAVE PÚBLICA.....	4
3.2- JERARQUÍA DE AUTORIDADES DE CERTIFICACIÓN.....	5
3.3- EXPLICACIÓN DE LA ELECCIÓN DE LA CONFIGURACIÓN.....	5
3.4- DESARROLLO DE LA IMPLEMENTACIÓN.....	7
3.5- MOMENTO DE UTILIZACIÓN DE CERTIFICADOS Y SU FUNCIONALIDAD.....	8
4- PRUEBAS.....	9
ANEXO 1: EVIDENCIA DE LAS PRUEBAS.....	12

1- PROPÓSITO DEL PROYECTO

El objetivo de la aplicación es proporcionar un espacio de mensajería segura entre usuarios, donde las credenciales (contraseña del usuario) y los mensajes estén protegidos mediante las técnicas criptográficas vistas en clase. Los usuarios pueden registrarse o iniciar sesión, gestionar su lista de contactos (ver o añadir contactos), enviar mensajes de forma confidencial y auténtica y borrar mensajes tanto recibidos como enviados. Todos los mensajes intercambiados están protegidos criptográficamente: las contraseñas de los usuarios se almacenan en hashes (bcrypt) y los mensajes se cifran de forma simétrica usando el algoritmo AES-256 en modo EAX, garantizando así la integridad y la privacidad de la información tanto almacenada como transmitida.

2- UTILIDAD FIRMA DIGITAL, ALGORITMOS UTILIZADOS Y GESTIÓN Y ALMACENAMIENTO DE CLAVES Y FIRMAS

2.1- UTILIDAD FIRMA DIGITAL

La firma digital se utiliza para mejorar la seguridad y el cifrado. El cifrado AES asegura la confidencialidad del contenido, restringiendo el acceso a emisor y receptor. Sin embargo, este método no autentica al remitente ni verifica la integridad del mensaje, esto lo conseguimos con la firma digital.

En nuestra aplicación, un usuario tiene la opción de firmar sus mensajes usando su clave privada RSA. Primero, se obtiene un resumen mediante SHA-256 y después se calcula la firma sobre el contenido del mensaje. Dicho valor, junto con un indicador de la firma, se agrega al mensaje antes de almacenarlo en el archivo JSON de mensajes.

Al recibir un mensaje firmado, la aplicación permite verificar la firma con la clave pública del remitente. Este proceso asegura que:

- El mensaje proviene del dueño de la clave privada correspondiente a la clave pública (autenticidad).
- El mensaje no ha sido modificado desde su firma (integridad).
- El remitente no puede negar haber enviado el mensaje, puesto que solo su clave privada pudo generar la firma (no repudio).

En resumen, la firma digital garantiza la confidencialidad de los mensajes, permite atribuirlos al remitente y evita manipulaciones no deseadas.

2.2- ALGORITMOS UTILIZADOS

En nuestra aplicación utilizamos algoritmos de cifrado simétricos y asimétricos estándar y altamente usados:

RSA (2048 bits) para la generación de claves asimétricas y la firma digital. Hemos elegido RSA por su amplio uso, buen soporte en las bibliotecas de python y seguridad adecuada con claves de 2048 bits. Además, se alinea con el modelo de “clave privada para firmar” y “clave pública para verificar”.

SHAS-256 como función hash para resumir los mensajes que se firman. SHA-256 es resistente a colisiones e imágenes previas, lo que la hace adecuada para ser una base de un esquema de firma digital seguro. En lugar de calcular la firma directamente sobre el mensaje, se calcula sobre su hash, siguiendo un patrón común de “hash + firma”.

PKCS#1 v1.5 como esquema de padding en las operaciones de firma con RSA. Este método está implementado de forma directa en las bibliotecas utilizadas, lo que agiliza la implementación, reduciendo los riesgos de error al crear firmas manualmente.

AES-256 en modo EAX para el cifrado de los mensajes. Aunque no participa directamente en la firma, AES-256-EAX asegura la confidencialidad e integridad autenticada del mensaje cifrado. EAX, al ser un modo AEAD (cifrado autenticado con datos asociados), defiende tanto el contenido como su integridad contra modificaciones.

La combinación de RSA + SHA-256 para firmas y AES-256-EAX para cifrado ofrece un equilibrio entre seguridad, claridad y sencillez en nuestra aplicación. El proceso consiste en firmar el mensaje (RSA+SHA-256) y luego cifrarlo (AES-256-EAX). El receptor, por su parte, primero descifra el mensaje y luego verifica la firma usando la clave pública del emisor.

2.3- GESTIÓN Y ALMACENAMIENTO DE CLAVES Y FIRMAS

La administración de claves y firmas se organiza en una estructura definida, que separa las funciones del usuario, las operaciones criptográficas y el almacenamiento.

1. Generación de claves asimétricas por el usuario:

Cada usuario tiene un par de claves RSA únicas, que se crean durante el registro inicial o al iniciar sesión si no existen claves previas. Este proceso tiene lugar localmente en el dispositivo del usuario, empleando un generador de números aleatorios seguro para fines criptográficos. Las claves tienen una longitud de 2048 bits, lo que asegura una seguridad adecuada.

2. Protección y almacenamiento de la clave privada:

- Se requiere que el usuario cree una contraseña de al menos 8 caracteres para salvaguardar su clave privada, confirmando su exactitud durante el proceso de configuración.
- La clave privada se cifra usando un esquema de derivación de clave tipo PBKDF2, junto con un cifrado simétrico fuerte (como AES-256), para proteger el archivo PEM guardado en el disco.
- La contraseña no se guarda de forma permanente; solo se usa en la memoria para obtener la clave necesaria para descifrar la clave privada cuando se requiere firmar.

Así, el acceso no autorizado al archivo de la clave privada no comprometerá su seguridad, ya que se necesita la contraseña asociada para poder usarla.

3. Almacenamiento de la clave pública y certificados:

La clave pública de cada usuario se guarda sin cifrar, usualmente en formato PEM, y se relaciona con un certificado “x509” emitido por una Autoridad de Certificación subordinada dentro de la PKI definida. Estos certificados vinculan la identidad (correo del usuario, nombre) con su clave pública de manera confiable. Los ficheros de certificados y configuración de la PKI se organizan en una estructura de directorios (dentro de la carpeta pki tenemos: ca_raíz, ca_subordinada, usuarios), con separación entre AC raíz, la subordinada y los certificados de usuario.

4. Gestión y almacenamiento de firmas:

Cuando un usuario opta por firmar un mensaje:

- El sistema carga su clave privada desde el disco, solicitando la contraseña para su descifrado.

- Se calcula el hash del mensaje usando SHA-256
- La firma se genera con RSA y se codifica en base64 para su almacenamiento seguro en formato JSON

El archivo de mensajes (mensajes.json), cada entrada incluye, además de los campos comunes (de, para, mensaje cifrado y clave simétrica cifrada/codificada) los siguientes campos relacionados con la firma:

- firmado: Un valor booleano que indica si el mensaje ha sido firmado.
- firma: la firma digital en base64, presente solo si firmado es *True*

Al momento de la lectura, si el receptor quiere verificar la firma:

- Se carga la clave pública del remitente, ya sea desde su par de claves o certificado.
- Se vuelve a calcular el hash del mensaje
- Este resultado se compara con la firma guardada usando RSA y SHA-256

Si ambos coinciden, la firma se considera válida, y el sistema notifica al usuario que el mensaje es auténtico e íntegro. De lo contrario, se notifica que la firma es inválida o que el mensaje pudo haber sufrido modificaciones.

3- GENERACIÓN CERTIFICADOS DE CLAVE PÚBLICA, JERARQUÍA DE AUTORIDADES DE CERTIFICACIÓN E IMPLEMENTACIÓN

3.1- GENERACIÓN CERTIFICADOS CLAVE PÚBLICA

En nuestra aplicación, la creación de certificados de clave pública sigue un proceso ordenado. Este proceso conecta la identidad de cada usuario con su clave pública mediante una firma digital emitida por una autoridad de certificación. El proceso empieza cuando un usuario se registra o entra en el sistema por primera vez y no tiene credenciales de seguridad. En este momento, el sistema crea automáticamente un par de claves RSA de 2048 bits, empleando un generador de números aleatorios seguro para asegurar que la clave privada sea única y no se pueda predecir.

Una vez creadas las claves, la clave pública sirve como base para la elaboración del certificado "x509". Este certificado digital es una estructura de datos que consta de varios campos importantes: la identidad del propietario de la clave (con información como su correo electrónico y nombre), la clave pública correspondiente, la identidad del emisor (la Autoridad de Certificación Subordinada) un número de serie único que identifica el certificado en el ámbito de la AC y un periodo de validez que establece los límites de confianza en el certificado. Por otra parte, se adjuntan extensiones "x509" que detallan los usos permitidos de la clave, como firma digital, no repudio y cifrado de claves.

El proceso de firma del certificado es crucial para establecer la confianza en la vinculación entre una identidad y una clave pública. La Autoridad de Certificación subordinada toma los campos del certificado y calcula un resumen criptográfico (hash). Este hash se firma digitalmente con clave privada RSA de la AC, creando así la firma del certificado. Esta firma se añade al certificado junto con detalles sobre el algoritmo utilizado para firmar. El resultado es un certificado completo que se puede verificar obteniendo la clave pública de la AC subordinada y comprobando la validez de la firma del certificado.

El certificado generado se guarda en formato PEM dentro de la carpeta de *pki* → *usuarios* bajo un nombre que identifica al usuario. A la vez, se actualiza un registro JSON (en *pki* → *config.json*) con un listado de todos los certificados emitidos, incluyendo datos como el correo, la fecha de emisión y el

número de serie. Este proceso automático garantiza que cada usuario tenga un certificado válido para autenticar su clave pública al firmar mensajes o establecer comunicaciones seguras.

3.2- JERARQUÍA DE AUTORIDADES DE CERTIFICACIÓN

El sistema emplea una infraestructura de clave pública (PKI) organizada en dos niveles: una Autoridad de Certificación raíz y una Autoridad de Certificación subordinada. Esta configuración refleja la estructura jerárquica habitual en implementaciones reales de PKI, buscando un equilibrio entre seguridad, escalabilidad y simplicidad de gestión.

En el nivel más alto de la jerarquía se encuentra la Autoridad de Certificación raíz, que asegura la confianza en toda la infraestructura. Su certificado está autofirmado, indicando que la AC raíz es tanto el emisor como el sujeto. Esta autoridad tiene una tarea importante, aunque limitada: emitir y firmar certificados de autoridades subordinadas, pero no de usuarios finales. Su clave privada se protege cuidadosamente y se usa solo para operaciones de alto nivel. El certificado de la AC raíz incluye información que especifica su rol como autoridad certificadora, limitando el número de niveles jerárquicos permitidos.

El segundo nivel lo ocupa la Autoridad de Certificación subordinada, cuya validez proviene directamente de la AC raíz. El certificado de esta autoridad, emitido y firmado digitalmente por la AC raíz, establece una relación jerárquica clara. La AC subordinada se encarga de las operaciones diarias de la PKI, incluyendo la expedición de certificados a los usuarios finales. Su certificado indica que es una autoridad certificadora, aunque con restricciones que impiden la creación de nuevas AC subordinadas, limitando la profundidad de la jerarquía.

En la base de la jerarquía están los usuarios finales. Cada usuario registrado en la aplicación recibe un certificado “x509”, expedido por la AC subordinada. Estos certificados incluyen la extensión *BasicConstraints* con CA= FALSE, lo que indica que no son autoridades certificadoras y que solo pueden usarse para la firma digital y cifrado. La cadena de confianza completa consta de tres partes: certificado de usuario, certificado de AC subordinada y certificado de AC raíz autofirmado. Esta estructura permite validar cualquier certificado de usuario al verificar las firmas de la cadena hasta llegar al certificado raíz, en el cual el sistema debe confiar.

3.3- EXPLICACIÓN DE LA ELECCIÓN DE LA CONFIGURACIÓN

La opción de ejecutar una jerarquía PKI de dos niveles (raíz + subordinada), en lugar de otras posibilidades, se basa en un análisis detallado de las necesidades del proyecto, normas de seguridad y aspectos prácticos de ejecución y soporte.

Análisis de las alternativas descartadas:

Desde la perspectiva de la implementación, una jerarquía simple con una sola Autoridad de Certificación (AC) que emitiera directamente todos los certificados de usuario sería la opción más sencilla, pues sólo requeriría la gestión de un par de claves y un certificado raíz. Sin embargo, esta simplicidad presenta serios problemas de seguridad. La clave privada de la AC única se usaría continuamente para firmar certificados de usuario, lo que aumentaría su exposición a posibles riesgos. Cada operación de emisión de certificado requeriría acceso a la clave más importante del sistema. Si esta clave se viera comprometida, toda la infraestructura de confianza se vendría abajo al instante sin posibilidad de recuperación parcial. La revocación de esta AC implicaría reemplazar por completo la infraestructura desde el principio, lo que impactaría a todos los usuarios al mismo tiempo.

En el otro extremo, una jerarquía multinivel, con varias autoridades subordinadas, daría gran flexibilidad organizativa. Por ejemplo, podría haber una AC raíz, AC regionales subordinadas, AC departamentales subordinadas y, finalmente, emisores de certificados de usuario. Esta estructura permitiría delegar responsabilidades de manera muy detallada y aislar problemas de seguridad en partes específicas. Sin embargo, para una aplicación de mensajería de alcance limitado, esto añadiría complejidad innecesaria. Cada nivel extra suma pasos a la validación de certificados, aumenta las necesidades de almacenamiento y gestión, complica la congestión del ciclo de vida de varias AC y expone más puntos débiles sin ventajas reales en este caso.

Un modelo de confianza descentralizado como la Web de Confianza popularizada por PGP, elimina las autoridades centrales. En este esquema, cada usuario certifica las claves de otros, formando una red de confianza distribuida. Aunque este enfoque es atractivo debido a su descentralización, traslada la gestión de la confianza a los usuarios finales, quienes a menudo carecen de las habilidades técnicas requeridas. Esto empeora la experiencia del usuario, ya que deben entender conceptos complejos. La escalabilidad también es un problema, ya que la verificación de claves precisa de encontrar rutas de confianza a través de la red, lo cual aumenta la complejidad computacional. Este modelo puede no ser apropiado para una empresa o institución que busque control centralizado y políticas consistentes.

Ventajas de la configuración elegida:

La jerarquía de dos niveles combina lo mejor de ambos enfoques, separando claramente las responsabilidades. La Autoridad de Certificación Raíz (AC Raíz) se mantiene fuera de línea, usándose solo para certificar autoridades subordinadas en operaciones clave. Su clave privada se protege con fuertes medidas de seguridad, como hardware especializado y controles de acceso estrictos. Por otro lado, la AC Subordinada está en línea y gestiona la emisión continua de certificados de usuario. Su clave, aunque protegida, es más accesible. Esta estructura permite que, si la AC Subordinada se ve comprometida, la raíz pueda revocar el certificado comprometido y emitir uno nuevo para una AC Subordinada segura, manteniendo la confianza en la raíz.

La escalabilidad a futuro representa otra ventaja. Si la aplicación se expande o se diversifica, es fácil agregar autoridades subordinadas sin cambiar la raíz o los certificados de usuario ya existentes. Se pueden crear subordinadas especializadas por región, tipo de usuario, nivel de seguridad, o cualquier otro criterio de la organización. Cada subordinada puede tener otro criterio de la organización. Cada subordinada puede tener diferentes políticas de emisión adaptadas a su contexto, mientras mantiene la cadena de confianza hacia la raíz común.

Desde la gestión de incidentes, esta estructura permite respuesta por niveles. Un problema de seguridad en la subordinada puede solucionarse sin afectar a la raíz. Ante la sospecha de un ataque, pero sin confirmación, se podría rotar preventivamente la subordinada. Solo si la raíz se ve comprometida, sería necesario reconstruir la PKI por completo. La arquitectura de dos niveles simplifica este proceso al separar claramente las funciones raíz y operativas.

Finalmente, la configuración seleccionada se alinea con las prácticas comunes de la industria. Las principales infraestructuras de clave pública comerciales, tales como DigiCert, Let's Encrypt y las autoridades gubernamentales, emplean estructuras jerárquicas donde la raíz goza de gran estabilidad, mientras que las entidades intermedias o subordinadas gestionan las operaciones. Esta familiaridad simplifica el mantenimiento, permite la aplicación de conocimiento bien documentado y garantiza la interoperabilidad con herramientas y sistemas externos.

3.4- DESARROLLO DE LA IMPLEMENTACIÓN

La PKI del proyecto se implementó usando el módulo *pki.py*, que incluye la lógica para generar y manejar las autoridades de certificación, así como la emisión de certificados. La estructura está organizada en tres capas funcionales distintas.

Estructura de directorios y configuración.

Al iniciar la aplicación, la clase *GestorPKI* comprueba la existencia de la estructura de directorios de la PKI. De no existir, se generan de forma automática tres directorios principales dentro de la carpeta *pki*: *ca_raiz*, que guarda el certificado y la clave privada de la AC raíz; *ca_subordinada*, destinada a la AC subordinada; y *usuarios*, para los certificados de los usuarios finales. Adicionalmente, se genera un archivo de configuración, *pki_config.json*, que contiene metadatos sobre la PKI, incluyendo información de ambas autoridades y un registro exhaustivo de todos los certificados emitidos.

Estructura de directorios y configuración.

El método *generar_ac_raiz()* crea la autoridad de certificación (AC) raíz empleando RSA con claves de 2048 bits. Para ello, usa la biblioteca *cryptography* de Python con el propósito de construir un certificado “x509” v3 autofirmado. El proceso abarca: la generación segura del par de claves; la construcción del nombre distinguido (DN) del sujeto/emisor; y la inclusión de extensiones críticas como *BasicConstraints* (que indica que puede certificar a otros usuarios con *path_length=1*), *KeyUsage* (que permite la firma de certificados y listas de revocación) y *SubjectKeyIdentifier*. La clave privada se protege mediante cifrado con PBKDF2 y AES-256, usando una contraseña interna, tanto la clave como el certificado se serializan en formato PEM. Adicionalmente, el método registra toda la operación en los logs criptográficos del sistema.

De forma análoga, *generar_ac_subordinada()* crea el certificado de la AC subordinada, pero a diferencia del anterior, este no se autofirma. Carga el certificado y la clave privada de la AC raíz (desencriptando esta última con la contraseña correspondiente), construye un nuevo certificado de la subordinada con la clave privada de la raíz. Las extensiones se configuran apropiadamente con *BasicConstraints* con *path_length=0* para evitar la creación de más AC.

Emisión de certificados de usuario.

El método *emitir_certificado_usuario(correo, nombre, clave_publica_pem)* se encarga de crear certificados para los usuarios. Primero, carga el certificado y la clave privada de la autoridad certificada subordinada. Luego, incorpora la clave pública del usuario (generada antes en *gestion_claves.py*) y construye un certificado “x509” que identifica al usuario.

El certificado incluye extensiones como: *BasicConstraints* con *ca=False* (lo que indica que no es una autoridad certificadora), *KeyUsage* que se limita a firma digital, cifrado de clave y no repudio, *ExtendedKeyUsage* para proteger el correo, y *SubjectAlternativeName* con el correo del usuario.

El certificado se firma digitalmente con la clave privada de la autoridad certificadora subordinada, usando SHA-256. Al final, el certificado se guarda en formato PEM, se almacena en el directorio de usuarios, y se guarda la información importante (como el número de serie y la fecha de emisión) en el archivo de configuración para fines de auditoría.

Validación e integración.

El módulo incorpora también dos métodos de validación esenciales. El método *verificar_firma_criptografica()* verifica, mediante cálculos matemáticos, que un certificado haya sido firmado de forma correcta por la clave pública de su emisor. El método *verificar_cadena_certificacion()*, por su parte, comprueba la totalidad de la cadena, desde el

certificado del usuario hasta la Autoridad de Certificación raíz. Este proceso asegura que cada eslabón haya sido firmado de modo correcto y que los certificados se encuentren dentro de su periodo de validez. Estos métodos se activan cuando el receptor busca verificar la autenticidad del mensaje firmado.

En *main.py*, la integración con el resto de la aplicación se lleva a cabo cuando un usuario se registra o inicia sesión. En ese momento, la función *configurar_claves_y_certificado()* que genere el certificado correspondiente. Así, la PKI se inicializa de forma automática en la primera ejecución y cada usuario consigue su certificado sin la necesidad de intervención manual.

3.5- MOMENTO DE UTILIZACIÓN DE CERTIFICADOS Y SU FUNCIONALIDAD

Los certificados de clave pública cumplen dos funciones esenciales en la aplicación, activándose en momentos clave del ciclo de vida de los mensajes firmados y verificados.

Verificación de firmas digitales en la recepción de mensajes.

Cuando un usuario recibe un mensaje con firma digital, el certificado del remitente permite verificar su autenticidad. El método *ver_mensajes_recibidos()* de la clase *GestorMensajes*, si el receptor opta por verificar una firma tras descifrar el mensaje, el sistema busca el certificado del remitente dentro de la carpeta *pki* en *usuarios*. Con este certificado, extrae la clave pública del remitente y comprueba que el certificado sea legítimo, verificando su cadena de confianza (remitente → AC subordinada → AC raíz) a través de *verificar_cadena_certificacion()*. Tras confirmar la validez del certificado y la vinculación de la firma al certificado del remitente, usa la clave pública para verificar la firma RSA sobre el hash SHA-256 del mensaje. Si ambas verificaciones son correctas, el usuario confirma que el mensaje proviene del remitente no puede negar su envío.

Establecimiento de la cadena de confianza y modelo de identidad.

Los certificados construyen un sistema de identidad apoyado en una jerarquía de confianza. Cada certificado de usuario une de forma criptográfica la identidad de la persona (correo, nombre) con su clave pública a través de la firma de una autoridad certificadora. Esto implica que un usuario no necesita intercambiar claves públicas mediante canales seguros alternos; es suficiente obtener el certificado del otro usuario (que puede estar guardado localmente o compartido públicamente) para tener seguridad criptográfica de que esa clave pública le corresponde. La cadena de usuario → AC subordinada → AC raíz ofrece una ruta de confianza que permite verificar la relación identidad-clave de cualquier usuario, siempre y cuando se confíe en la AC raíz.

Control de acceso y autorización.

En esencia, los certificados sirven como un sistema de autorización. Solo aquellos que han pasado por el registro en la aplicación y tienen un certificado válido de la AC subordinada pueden usar completamente el sistema de mensajería firmada. Tener un certificado válido muestra que la aplicación ha reconocido al usuario. En el futuro, la información en las extensiones del certificado (como *ExtendedKeyUsage*) podría servir para controlar qué operaciones criptográficas puede hacer cada usuario (firmar, cifrar, etc).

Auditoría y no repudio reforzado.

Los certificados simplifican la auditoría mediante registros criptográficos. Cada vez que se emite o verifica un certificado, se guarda información clave como el número de serie, el nombre del usuario y la cadena de confianza validada. Esto genera un historial que permite saber quién tenía confianza validada. Esto genera un historial que permite saber quién tenía certificados válidos en un momento dado. Al combinarse con las firmas digitales de los mensajes, se obtiene una prueba legal de autoría. Si

un usuario niega haber enviado un mensaje firmado, el certificado, la cadena de confianza y la firma sirven como evidencia criptográfica de su responsabilidad.

4- PRUEBAS

Prueba 1: Inicialización de la PKI al ejecutar por primera vez

- *Descripción:* Se ejecuta la aplicación por primera vez
- *Salida esperada:*
Por terminal:
 - [PKI] Generando Autoridad de Certificación Raíz...
 - [PKI] ✓ AC Raíz creada exitosamente
 - [PKI] Generando Autoridad de Certificación Subordinada...
 - [PKI] AC Subordinada creada y certificada por AC Raíz
- Logs [CRYPTO] contienen:
 - CREACIÓN AC RAÍZ | Algoritmo: RSA | Tamaño clave: 2048 bits
 - CREACIÓN AC SUBORDINADA | Certificado: Firmado por AC Raíz
- Evidencia: Ver Anexo I, Figura 1

Prueba 2: Verificación de estructura de directorios PKI

- *Descripción:* Tras la inicialización, se comprueba que existen las carpetas y archivos de la PKI
- *Salida esperada:* se crean los archivos:
Dentro de la carpeta PKI:
 - ca_raiz/ca_raiz_cert.pem y ca_raiz_private.pem
 - ca_subordinada/ca_sub_cert.pem y ca_sub_private.pem
 - pki_config.json con la configuración de las AC
- Evidencia: Ver Anexo I, Figura 2

Prueba 3: Emisión de certificado al registrar a un usuario

- *Descripción:* Se registra un nuevo usuario (pepa@gmail.com) desde el menú principal
- *Salida esperada:*
Por terminal:
 - Usuario 'Pepa' registrado correctamente
 - CONFIGURACIÓN DE CREDENCIALES DE SEGURIDAD
 - Clave privada generada y guardada
 - Clave pública generada
 - [PKI] Certificado emitido para Pepa (pepa@gmail.com)
 - Archivo: pki/usuarios/pepa_at_gmail_com_cert.pem
- Logs [CRYPTO] contienen:
 - GENERACIÓN DE CLAVES RSA | Algoritmo: RSA | Tamaño: 2048 bits
 - EMISIÓN CERTIFICADO USUARIO | Usuario: Pepa (pepa@gmail.com |
 - Emisor: AC Subordinada
- Evidencia: Ver Anexo I, Figura 3

Prueba 4: Registro de certificado en pki_config.json

- *Descripción:* Se verifica que el certificado emitido quedó registrado en el archivo de configuración
- *Salida esperada:* en pki_config.json, dentro de certificados emitidos aparece la entrada con los datos
- Evidencia: Ver Anexo I, Figura 4

Prueba 5: Registro de varios usuarios y verificación de múltiples certificados

- *Descripción:* Se registran 2 usuarios más (lola y raquel)
- *Salida esperada:* en pki/usuarios aparecen los 3 certificados:
 - pepa_at_gmail_com_cert.pem
 - lola_at_gmail_com_cert.pem
 - raquel_at_gmail_com_cert.pem
- Evidencia: Ver Anexo I, Figura 5

Prueba 6: Agregar contacto entre 2 usuarios

- *Descripción:* Pepa agrega a Lola como contacto
- *Salida esperada:* Por terminal:
 - ¡Bienvenido/a, Pepa García!
 - Contacto agregado correctamente: Lola Martínez (lola@gmail.com)
- En json/contactos.json, en la entrada de pepa@gmail.com, aparece lola@gmail.com en su lista de contactos
- Por terminal:
 - Contacto agregado correctamente
- En contactos.json, en la entrada de pepa@gmail.com, aparece lola@gmail.com en su lista de contactos
- Evidencia: Ver Anexo I, Figura 6

Prueba 7: Envío de mensaje sin firmar

- *Descripción:* Pepa envía un mensaje a lola sin firmar
- *Salida esperada:*
 - Por terminal:
 - Logs [CRYPTO]:
 - GENERACIÓN CLAVE AES | Algoritmo: AES-256
 - CIFRADO DE MENSAJE | Modo: EAX (Cifrado Auténtico AEAD)
 - ✓ Mensaje enviado correctamente. (sin mención de firma)
- En mensajes.json, la entrada tiene “firmado”: false y no existe campo “firma”
- Evidencia: Ver Anexo I, Figura 7

Prueba 8: Lectura de mensaje sin firma

- *Descripción:* Lola lee el mensaje enviado por pepa sin firma
- *Salida esperada:*
 - Por terminal:
 - De: pepa@gmail.com | 2025-12-03 13:XX:XX
 - Mensaje: envío de mensaje sin firma
 - NO aparece mensaje de verificación de firma
 - Logs [CRYPTO]:
 - DESCIFRADO DE MENSAJE | Estado verificación: ✓ CORRECTA
 - NO aparece VERIFICACIÓN DE FIRMA DIGITAL
- Evidencia: Ver Anexo I, Figura 8

Prueba 9: Envío de mensaje con firma digital

- *Descripción:* Pepa envía un mensaje a Lola firmado digitalmente
- *Salida esperada:*
Por terminal:
 - Mensaje firmado digitalmente
 - Mensaje enviado correctamente y firmado.Logs [CRYPTO]:
 - CARGA CLAVE PRIVADA | Estado: Clave privada descifrada exitosamente.
 - GENERACIÓN DE FIRMA DIGITAL | Algoritmo: RSA con PKCS#1 v1.5, Hash: SHA-256
 - GENERACIÓN CLAVE AES y CIFRADO DE MENSAJE (igual que en mensaje normal)En mensajes.json, la entrada del mensaje tiene “firmado”: true y un campo “firma”.
- Evidencia: Ver Anexo I, Figura 9

Prueba 10: Lectura y verificación de mensaje con firma

- *Descripción:* Lola lee el mensaje firmado de Pepa y verifica la firma
- *Salida esperada:*
Logs [CRYPTO]:
 - DESCIFRADO DE MENSAJE | Estado verificación: ✓ CORRECTA
 - VERIFICACIÓN DE FIRMA DIGITAL | Algoritmo: RSA con PKCS#1 v1.5, Hash: SHA-256
- Evidencia: Ver Anexo I, Figura 10

Prueba 12: Manipulación de mensaje y detección de integridad rota

- *Descripción:* Se edita manualmente *mensajes.json* para romper la integridad del mensaje
- *Salida esperada:*
Por terminal: Mensaje de error.
Logs [CRYPTO]:
 - CARGA CLAVE PRIVADA
 - ERROR DESCIFRADO
- Evidencia: Ver Anexo I, Figura 12

Prueba 13: Manipulación de firma y detección de firma inválida

- *Descripción:* Se edita manualmente *mensajes.json* para romper solo el campo de firma
- *Salida esperada:*
Por terminal: Se muestra el mensaje y el mensaje de error
Logs [CRYPTO]:
 - DESCIFRADO DE MENSAJE | Estado: ✓ CORRECTA
 - ERROR VERIFICACIÓN FIRMA → Firma inválida
- Evidencia: Ver Anexo I, Figura 13

Prueba 14: Intento de iniciar sesión con contraseña incorrecta

- *Descripción:* Se intenta iniciar sesión como usuario con contraseña incorrecta
- *Salida esperada:* Correo o contraseña incorrectos
Logs [CRYPTO]:
 - AUTENTICACIÓN USUARIO | Operación: Verificación de contraseña | Resultado: ✗ Incorrecta
- Evidencia: Ver Anexo I, Figura 14

ANEXO 1: EVIDENCIA DE LAS PRUEBAS

Figura 1. Inicialización de la PKI al ejecutar por primera vez

```
(base) C:\Users\raque\OneDrive\Escritorio\02_E1_app>main.py

[PKI] Generando Autoridad de Certificación Raíz...

[CRYPTO] 2025-11-30 01:18:28 | CREACIÓN AC RAÍZ
  - Tipo: Autoridad de Certificación Raíz (Root CA)
  - Nombre: AC Raíz - Mensajería Segura UC3M
  - Algoritmo clave: RSA
  - Tamaño clave: 2048 bits
  - Algoritmo firma: SHA-256 con RSA
  - Validez: 20 años
  - Certificado: Autofirmado
  - Uso: Firma de certificados de ACs subordinadas
  - Path length: 1 (puede certificar 1 nivel de ACs)

[PKI] AC Raíz creada exitosamente
  Certificado: pki\ca_raiz\ca_raiz_cert.pem
  Clave privada: pki\ca_raiz\ca_raiz_private.pem (protegida con contraseña)

[PKI] Generando Autoridad de Certificación Subordinada...

[CRYPTO] 2025-11-30 01:18:28 | CREACIÓN AC SUBORDINADA
  - Tipo: Autoridad de Certificación Subordinada (Intermediate CA)
  - Nombre: AC Subordinada - Usuarios Mensajería
  - Algoritmo clave: RSA
  - Tamaño clave: 2048 bits
  - Algoritmo firma: SHA-256 con RSA
  - Validez: 10 años
  - Certificado: Firmado por AC Raíz
  - Uso: Firma de certificados de usuarios finales
  - Path length: 0 (NO puede certificar otras ACs)

[PKI] AC Subordinada creada y certificada por AC Raíz
  Certificado: pki\ca_subordinada\ca_sub_cert.pem
  Clave privada: pki\ca_subordinada\ca_sub_private.pem (protegida con contraseña)
```

Figura 1.1. Inicialización cuando el árbol está previamente creado

```
(base) C:\Users\raque\OneDrive\Escritorio\02_E1_app>main.py

[PKI] La AC Raíz ya existe.

[PKI] La AC Subordinada ya existe.

=== APP DE MENSAJERÍA SEGURA ===
1. Registrarse
2. Iniciar sesión
3. Salir
Elige una opción escribiendo solo el número (ej: 1):
```


Figura 2. Verificación de estructura de directorios PKI



Figura 2.1. Verificación de los archivos PKI

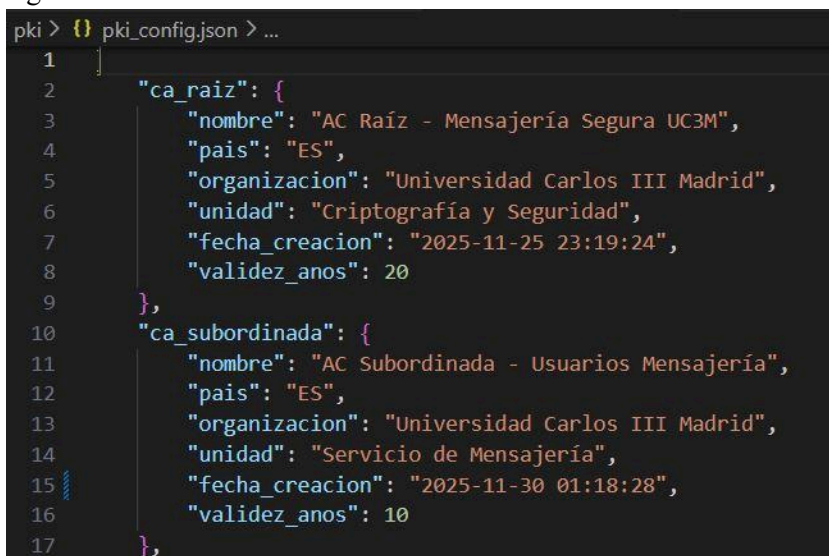
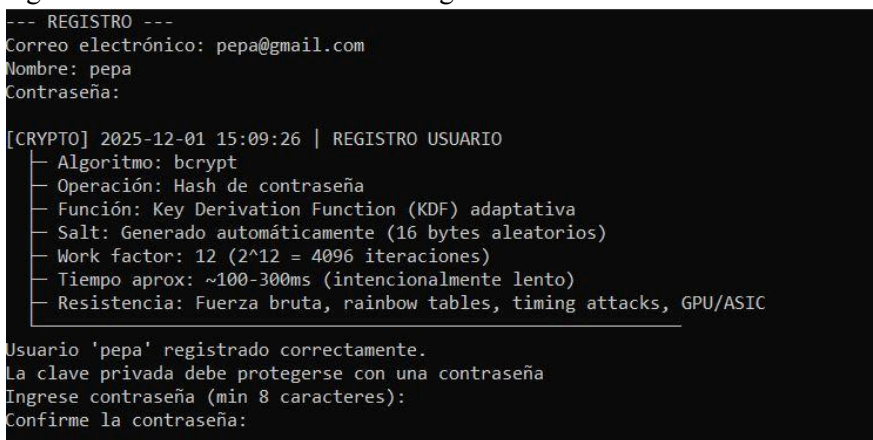


Figura 3. Emisión de certificado al registrar a un usuario




```
[CLAVES] Generando par de claves RSA para pepa...

[CRYPTO] 2025-12-01 15:09:49 | GENERACIÓN PAR CLAVES RSA
├─ Algoritmo: RSA
├─ Tamaño clave: 2048 bits
├─ Exponente público: 65537
├─ Uso: Firma digital + Cifrado asimétrico
├─ Seguridad: Nivel apropiado para uso actual (NIST recomienda ≥2048 bits)
├─ Componentes: Clave privada (d, n) + Clave pública (e, n)
└─ Generador: Crypto.PublicKey.RSA (PyCryptodome)

[CRYPTO] 2025-12-01 15:09:49 | ALMACENAMIENTO SEGURO CLAVES
├─ Usuario: pepa (pepa@gmail.com)
├─ Clave privada: claves_usuarios\pepa_at_gmail_com_private.pem
├─ Protección: [X] Cifrada con contraseña (PBKDF2 + AES-256-CBC)
├─ Clave pública: claves_usuarios\pepa_at_gmail_com_public.pem
├─ Formato: PEM (PKCS#8 para privada, X.509 SubjectPublicKeyInfo para pública)
└─ Seguridad: Acceso a clave privada requiere contraseña del usuario

[CLAVES] [X] Par de claves generado y almacenado
Clave privada (protegida): claves_usuarios\pepa_at_gmail_com_private.pem
Clave pública: claves_usuarios\pepa_at_gmail_com_public.pem

[CRYPTO] 2025-12-01 15:09:49 | EMISIÓN CERTIFICADO USUARIO
├─ Tipo: Certificado X.509 de usuario final
├─ Usuario: pepa (pepa@gmail.com)
├─ Emisor: AC Subordinada - Usuarios Mensajería
├─ Número de serie: 318257272600649426044960975345795047522448902997
├─ Algoritmo firma: SHA-256 con RSA
├─ Validez: 2 años
├─ Uso clave: Firma digital + Cifrado + No repudio
└─ Cadena confianza: Usuario → AC Subordinada → AC Raíz

[PKI] Certificado emitido para pepa (pepa@gmail.com)
Archivo: pki\usuarios\pepa_at_gmail_com_cert.pem

--- MENÚ DE USUARIO ---
1. Ver contactos
2. Añadir contacto (con validación PKI)
3. Enviar mensaje seguro
4. Ver mensajes recibidos
5. Cerrar sesión
Elige una opción escribiendo solo el número (ej: 1):
```

Figura 4. Registro de certificado en pki_config.json

```
{
  "correo": "pepa@gmail.com",
  "nombre": "pepa",
  "fecha_emision": "2025-12-01 15:09:49",
  "numero_serie": "318257272600649426044960975345795047522448902997",
  "archivo": "pki\\usuarios\\pepa_at_gmail_com_cert.pem"
}
```

Figura 5. Registro de varios usuarios y verificación de múltiples certificados

```
"certificados_emitidos": [
  {
    "correo": "raquel@gmail.com",
    "nombre": "raquel",
    "fecha_emision": "2025-11-25 23:29:35",
    "numero_serie": "401598888578096937895806531127389426740669523258",
    "archivo": "pki\\usuarios\\raquel_at_gmail_com_cert.pem"
  },
  {
    "correo": "lola@gmail.com",
    "nombre": "lola",
    "fecha_emision": "2025-11-26 01:14:41",
    "numero_serie": "120685361106042106027934467068012542788503286310",
    "archivo": "pki\\usuarios\\lola_at_gmail_com_cert.pem"
  },
  {
    "correo": "pepa@gmail.com",
    "nombre": "pepa",
    "fecha_emision": "2025-12-01 15:09:49",
    "numero_serie": "318257272600649426044960975345795047522448902997",
    "archivo": "pki\\usuarios\\pepa_at_gmail_com_cert.pem"
  }
]
```

Figura 6. Agregar contacto entre 2 usuarios

```
Correo del contacto a agregar: lola@gmail.com
Iniciando protocolo de seguridad para validar a lola@gmail.com.

[PKI] Iniciando verificación de cadena de certificación...
  Eslabón 1: Certificado Usuario firmado correctamente por AC Subordinada
  Eslabón 2: Certificado AC Subordinada firmado correctamente por AC Raíz

[CRYPTO] 2025-12-04 18:51:25 | VALIDACIÓN CADE CERTIFICACIÓN
  - Usuario: 1.2.840.113549.1.9.1=lola@gmail.com,CN=lola,OU=Usuarios Mensajería,O=Universidad Carlos III Madrid,C=ES
  - Estado: VÁLIDO
  - Cadena: Usuario -> AC Sub -> AC Raíz

  Cadena de certificación completa verificada

Contacto agregado correctamente.
```

Figura 6.1. Fichero con los contactos agregados

```
{
  "usuario": "pepa@gmail.com",
  "contactos": [
    {
      "correo": "lola@gmail.com",
      "ruta_publica": "pki\\usuarios\\lola_at_gmail_com_cert.pem",
      "nombre_archivo": "lola_at_gmail_com_cert.pem"
    }
  ]
},
{
  "usuario": "lola@gmail.com",
  "contactos": [
    {
      "correo": "pepa@gmail.com",
      "ruta_publica": "claves_usuarios\\pepa_at_gmail_com_public.pem",
      "nombre_archivo": "pepa_at_gmail_com_public.pem"
    }
  ]
}
```

Prueba 7: Envío de mensaje sin firmar

```
Tus contactos:
Correo del destinatario: lola@gmail.com
Escribe tu mensaje: envío de mensaje sin firma

¿Desea firmar digitalmente este mensaje? (si/no): no

[CRYPTO] 2025-12-04 18:53:42 | GENERACIÓN CLAVE AES
├─ Algoritmo: AES-256
├─ Tamaño clave: 256 bits (32 bytes)
├─ Generador: Crypto.Random.get_random_bytes (CSPRNG)
├─ Fuente entropía: Sistema operativo (/dev/urandom o CryptGenRandom)
└─ Seguridad: Criptográficamente seguro e impredecible

[CRYPTO] 2025-12-04 18:53:42 | CIFRADO DE MENSAJE
├─ Algoritmo: AES-256-EAX
├─ Modo: EAX (Cifrado Autenticado AEAD)
├─ Tamaño clave: 256 bits (32 bytes)
├─ Tamaño nonce: 16 bytes
├─ Tamaño tag MAC: 16 bytes
├─ Tamaño mensaje original: 26 bytes
├─ Tamaño mensaje cifrado: 26 bytes
└─ Propiedades: Confidencialidad + Integridad + Autenticidad

Mensaje enviado correctamente.
```

Prueba 7.1. Mensaje guardado

```
{
  "de": "pepa@gmail.com",
  "para": "lola@gmail.com",
  "mensaje": {
    "nonce": "2mJw3S6V/EYVy1lZcXumDA==",
    "cifrado": "x4tIUrxR7Pkhnt6QzwxwVFXdHG/b4GAZyJo=",
    "etiqueta": "cToqN6YJX+2A6LvFbn/5vw=="
  },
  "clave_cifrada": "ZchlIDuUkUc40nTiYcrit8vuf7Z4qCfy1Jb68QGyp0JJ70UEphZ/P8d0fkJC/dnsu",
  "firmado": false,
  "fecha": "2025-12-04 18:53:42"
},
```

Figura 8. Lectura de mensaje sin firma

```
Para leer, introduce tu contraseña de clave privada:

[CRYPTO] 2025-12-04 20:37:10 | CARGA CLAVE PRIVADA
├─ Usuario: lola@gmail.com
├─ Estado: ✓ Clave privada descifrada exitosamente
├─ Archivo: claves_usuarios\lola_at_gmail_com_private.pem
└─ Protección: Contraseña verificada correctamente

--- MENSAJES RECIBIDOS ---

[CRYPTO] 2025-12-04 20:37:10 | DESCIFRADO DE MENSAJE
├─ Algoritmo: AES-256-EAX
├─ Operación: Descifrado y verificación de integridad
├─ Estado verificación: ✓ CORRECTA - Mensaje íntegro y auténtico
├─ Tamaño descifrado: 26 bytes
└─ Seguridad: Sin modificaciones detectadas

De: pepa@gmail.com | 2025-12-04 20:36:44
Mensaje: envío de mensaje sin firma
```


Figura 9. Envío de mensaje con firma digital

```
Tus contactos:
Correo del destinatario: lola@gmail.com
Escribe tu mensaje: mensaje firmado

¿Desea firmar digitalmente este mensaje? (si/no): si
Contraseña de su clave privada:

[CRYPTO] 2025-12-04 18:58:11 | CARGA CLAVE PRIVADA
- Usuario: pepa@gmail.com
- Estado: ☒ Clave privada descifrada exitosamente
- Archivo: claves_usuarios\pepa_at_gmail_com_private.pem
- Protección: Contraseña verificada correctamente

[CRYPTO] 2025-12-04 18:58:11 | GENERACIÓN DE FIRMA DIGITAL
- Algoritmo: RSA con PKCS#1 v1.5
- Hash: SHA-256
- Tamaño clave: 2048 bits
- Tamaño firma: 256 bytes (2048 bits)
- Tamaño mensaje: 15 bytes
- Tamaño hash: 256 bits (32 bytes)
- Propiedades: Autenticidad + Integridad + No repudio

Mensaje firmado digitalmente

[CRYPTO] 2025-12-04 18:58:11 | GENERACIÓN CLAVE AES
- Algoritmo: AES-256
- Tamaño clave: 256 bits (32 bytes)
- Generador: Crypto.Random.get_random_bytes (CSPRNG)
- Fuente entropía: Sistema operativo (/dev/urandom o CryptGenRandom)
- Seguridad: Criptográficamente seguro e impredecible

[CRYPTO] 2025-12-04 18:58:11 | CIFRADO DE MENSAJE
- Algoritmo: AES-256-EAX
- Modo: EAX (Cifrado Autenticado AEAD)
- Tamaño clave: 256 bits (32 bytes)
- Tamaño nonce: 16 bytes
- Tamaño tag MAC: 16 bytes
- Tamaño mensaje original: 15 bytes
- Tamaño mensaje cifrado: 15 bytes
- Propiedades: Confidencialidad + Integridad + Autenticidad

Mensaje enviado correctamente y firmado.
```

Figura 9.1. Mensaje guardado

```
{
  "de": "pepa@gmail.com",
  "para": "lola@gmail.com",
  "mensaje": {
    "nonce": "0X3rJDJlIkKkYmBoFQXRSQ==",
    "cifrado": "xKY3UQoVH04g+KstElCc",
    "etiqueta": "Z6fM3n7INs3ZHyeeg0QNHQ=="
  },
  "clave_cifrada": "B0XnR9FVCjB/JlUXr9Pk50EA1Fv70H5zs4xRGkXKIcXt72V7qg5QLJYI5lU1FWokJ+F+Tk",
  "firmado": true,
  "fecha": "2025-12-04 18:58:11",
  "firma": "pn8oZx6EVLzs1DVEcfo80lnn52VPJXDJDd2Y3uQu9/P1XC/B1Yd3skt6pvsPFDn/3k3P/JkvU1MhZw"
}
```

Figura 10. Lectura y verificación de mensaje con firma

```
Para leer, introduce tu contraseña de clave privada:

[CRYPTO] 2025-12-04 21:36:20 | CARGA CLAVE PRIVADA
| Usuario: lola@gmail.com
| Estado: ✓ Clave privada descifrada exitosamente
| Archivo: claves_usuarios\lola_at_gmail_com_private.pem
| Protección: Contraseña verificada correctamente

--- MENSAJES RECIBIDOS ---

[CRYPTO] 2025-12-04 21:36:20 | DESCIFRADO DE MENSAJE
| Algoritmo: AES-256-EAX
| Operación: Descifrado y verificación de integridad
| Estado verificación: ✓ CORRECTA - Mensaje íntegro y auténtico
| Tamaño descifrado: 15 bytes
| Seguridad: Sin modificaciones detectadas

De: pepa@gmail.com | 2025-12-04 21:32:58
Mensaje: mensaje firmado
    El mensaje tiene firma digital. Verificando...

[CRYPTO] 2025-12-04 21:36:20 | VERIFICACIÓN DE FIRMA DIGITAL
| Algoritmo: RSA con PKCS#1 v1.5
| Hash: SHA-256
| Resultado: FIRMA VÁLIDA
| Estado: Mensaje auténtico e íntegro
| Tamaño clave: 2048 bits
| Garantías: Autenticidad + Integridad + No repudio

FIRMA VÁLIDA - Mensaje auténtico e íntegro
```

Figura 12. Manipulación de mensaje y detección de integridad rota

```
Para leer, introduce tu contraseña de clave privada:

[CRYPTO] 2025-12-04 21:59:15 | CARGA CLAVE PRIVADA
| Usuario: pepa@gmail.com
| Estado: ✓ Clave privada descifrada exitosamente
| Archivo: claves_usuarios\pepa_at_gmail_com_private.pem
| Protección: Contraseña verificada correctamente

--- MENSAJES RECIBIDOS ---

[CRYPTO] 2025-12-04 21:59:15 | ERROR DESCIFRADO
| Error: Invalid base64-encoded string: number of data characters (9) cannot be 1 more than a multiple of 4
| Tipo: Error

Error: El mensaje está corrupto o fue modificado
```

Figura 13. Manipulación de firma y detección de firma inválida

```
Para leer, introduce tu contraseña de clave privada:

[CRYPTO] 2025-12-04 22:06:10 | CARGA CLAVE PRIVADA
├─ Usuario: lola@gmail.com
├─ Estado: ✓ Clave privada descifrada exitosamente
├─ Archivo: claves_usuarios\lola_at_gmail_com_private.pem
├─ Protección: Contraseña verificada correctamente
└─

--- MENSAJES RECIBIDOS ---

[CRYPTO] 2025-12-04 22:06:10 | DESCIFRADO DE MENSAJE
├─ Algoritmo: AES-256-EAX
├─ Operación: Descifrado y verificación de integridad
├─ Estado verificación: ✓ CORRECTA - Mensaje íntegro y auténtico
├─ Tamaño descifrado: 10 bytes
├─ Seguridad: Sin modificaciones detectadas
└─

De: pepa@gmail.com | 2025-12-04 22:02:46
Mensaje: Hola Lola!
    El mensaje tiene firma digital. Verificando...

[CRYPTO] 2025-12-04 22:06:10 | ERROR VERIFICACIÓN FIRMA
├─ Error: Invalid base64-encoded string: number of data characters (345) cannot be 1 more than a multiple of 4
├─ Tipo: Error
└─

FIRMA INVÁLIDA - Mensaje comprometido o firma incorrecta
```

Figura 14. Intento de iniciar sesión con contraseña incorrecta

```
--- INICIO DE SESIÓN ---
Correo electrónico: lola@gmail.com
Contraseña: ■

[CRYPTO] 2025-12-04 19:01:16 | AUTENTICACIÓN USUARIO
├─ Algoritmo: bcrypt
├─ Operación: Verificación de contraseña
├─ Usuario: lola@gmail.com
├─ Resultado: ✗ Incorrecta
├─ Método: Comparación en tiempo constante (previene timing attacks)
└─

Correo o contraseña incorrectos.

=== APP DE MENSAJERÍA SEGURA ===
1. Registrarse
2. Iniciar sesión
3. Salir
Elige una opción escribiendo solo el número (ej: 1):
```